

Audion Get Tools - user guide

Audion Get Tools installs, updates, and removes Windows software through WinGet: you tick what you need and press one button. This guide covers what is worth knowing while working with it - how to fetch an installer without installing anything, how to sort out the Visual C++ runtimes, and how to use AI-assisted package planning.

The project layout, the services, and the development rules live in the README. That is the technical documentation; it is not repeated here.

Downloading without installing

Every program in the list has small buttons to the right of its name. They do not touch the checkbox: pressing a button neither selects nor clears the entry, so the usual workflow stays the same.

Button	What it does
Green down arrow	Downloads the program's installer into <code>output\Downloads</code> and installs nothing.
Crimson box	Downloads the zip archive or the standalone build - the one that needs no installation. It appears only for programs that really have such a build.
Blue up arrow	Opens the download page in your browser: the GitHub releases page, the TechPowerUp page, or the vendor's own site.

The crimson button shows up on about a quarter of the list - 30 programs. Every package was checked for an archive build, and the button stays hidden where there is none, so it is never a dead press.

Notepad++, Everything, OBS Studio, VLC, Telegram, FFmpeg, Node.js, MKVToolNix, Audacity, Sysinternals, Rufus, yt-dlp, SumatraPDF, Tabby, Rclone, and RClone Manager are among them.

Sometimes the vendor ships a build that never reaches the WinGet catalogue - Tabby and RClone Manager are two. The file is then taken straight from the release page, and the button behaves exactly as it always does. Both buttons draw from the same release, so the installer and the portable build are

always the same version. For RClone Manager it is also the newer one: the WinGet catalogue carries the installer alone, and it trails what the vendor has already published.

The same helps when WinGet simply cannot hand out a file: if the program has a GitHub release page, Audion Get Tools looks for the build there. Every check stays in place - other systems, the wrong architecture, checksums and debug files are refused - and when no single file clearly fits, the button says so instead of downloading something at random.

Why this is useful when a normal install is right there:

- an installer can be carried to another machine or kept for work without a network;
- some programs do more when installed from their own distributive than from their WinGet package. PowerShell from the Microsoft installer adds Explorer context-menu entries and keeps updating itself afterwards, while the WinGet build does not;
- on the download page you can pick a build by hand: a different architecture, a portable edition, an earlier release.

`Google Chrome (web installer)` has its own download button next to it - it fetches the same web installer without running it.

Where things land:

Folder	What is there
<code>output\Downloads</code>	Installers, loose - no subfolders. You pick one up, run it, and forget it.
<code>output\Portable\<Program></code>	Whatever runs without installation: archives and standalone builds, each in a folder named after the program - <code>output\Portable\Notepad++</code> , <code>output\Portable\Rufus</code> .
<code>output\Install\<Bundle></code>	Whatever installs into the system, for example <code>output\Install\MSVC All-in-One (TechPowerUp)</code> .

The folder is named the way the vendor writes the product name. No `Google.Chrome`, no underscores. Inside that folder everything keeps the names it arrived with.

The split follows what a download does, not its extension: the MSVC bundle installs runtimes into the system, so it lives in `Install` even though it arrives as a zip - unpacked next to its archive so `install_all.bat` is at hand.

All of it sits under `output` and is wiped with it when the workspace is cleaned - by design: a portable build should not carry gigabytes of installers around. Move what you need somewhere safe; the rest

can simply be downloaded again.

Visual C++ (MSVC) bundles

Old games and programs need Visual C++ runtimes from various years. This is usually the messiest part of a Windows system, so it is laid out in three places.

Where	What is there
MSVC runtime (2015+)	Official Microsoft packages, x86 and x64. A modern system needs nothing else.
Advanced: MSVC 2005-2013	Official packages, one per year and architecture - for ancient software only. They never report an update: the vendor froze those builds.
Classic scripts	Two ready-made bundles: MSVC All-in-One (TechPowerUp) and MSVC AIO (abbodi1406) . Each has a download button and a button that opens its release page.

The bundles beat installing one package at a time: a single pass covers every runtime and includes 2012, which has no WinGet package at all. They are assembled and kept current by well-known maintainers, so for the older family they are the better choice.

The [Check MSVC runtimes](#) action shows what is actually installed, which version is available, and whether anything needs doing. Start there when it is not clear what is missing.

Installing runtimes changes the system, so Audion Get Tools asks for confirmation first. The application has to run as administrator - without that the install does not start and you get a message saying so.

Removal is not split into groups: all nine runtimes go at once. That is deliberate - this set is safer to remove completely and reinstall clean than to repair piece by piece.

AI Package Planner

The Audion Get Tools section that uses an LLM to suggest Windows/WinGet packages, validate exact IDs, and execute selected actions through the normal protected Audion paths.

Purpose

[AI Package Planner](#) lets a user describe a software task in natural language, get a short package plan, validate suggestions with WinGet search, and then run only selected exact IDs.

The section is not a separate installer. It must not bypass the existing install, update, pin, check, or uninstall flows. The LLM helps draft and explain a reviewable plan.

Primary value:

- suggest packages from a human request;
- reduce WinGet package ID mistakes;
- show a plan before installation;
- support focused search and exact-ID actions;
- keep keys, models, and prompt templates inside the GUI-managed flow.

Main Workflow Model

The section has two tabs:

Tab	Purpose
Planner	AI task, instruction template, instruction editor, plan building, WinGet search, and selected exact-ID execution.
Models	OpenAI/Gemini provider selection, keys, models, manual overrides, reasoning, and model checks.

Top-right CTA in the section navigation:

RUN SELECTED WITH AI

It opens the final form for running selected packages from the last AI plan. This is the main user CTA after a plan has already been generated and reviewed.

Build plan stays as the first button in the Actions grid because it starts planning; it is not the final execution step.

Planner Fields

AI task

What to find or prepare right now.

Examples:

- Find Media Player Classic Black Edition with WinGet.
- Prepare a Windows machine for Dev/Media.
- Suggest tools for screen recording, editing, and video conversion.

This field is required for building an AI plan. Without it, the LLM does not know what to suggest.

Instruction template

A saved AI behavior template.

Selecting a template loads text into `AI planner instruction`. It does not scan installed packages, call the LLM, or install anything.

Buttons beside the select:

Icon	Action
<code>sync</code>	Refresh template list.
<code>save</code>	Save the current instruction to prompt cache.
<code>push_pin</code>	Pin the selected template.
<code>block</code>	Unpin the selected template.
<code>delete</code>	Delete the selected template from cache.
<code>backspace</code>	Clear the instruction editor.

AI planner instruction

Rules for how the AI should reason and format the plan.

Most users do not need to edit this field every time. They can select an instruction template or keep the default prompt.

The instruction defines behavior:

- propose options first;
- do not install anything during planning;
- validate or clarify WinGet IDs;
- mention risks and ambiguous choices;
- keep the answer short and practical.

Scan installed IDs

Sends already installed WinGet IDs to the LLM.

This helps avoid suggesting packages that are already present. Installed state is based only on the installed-ID scan, not on known Audion groups.

Send Audion known package groups

Sends curated project lists to the LLM as context.

Important: known Audion groups are options and presets, not installed-state proof.

Search candidates

How many `winget search` rows to collect for each suggested package.

Smaller values are faster and cleaner. Larger values are useful for ambiguous searches.

Planner Actions

Action	Behavior	Type
<code>Build plan</code>	Sends the task and instruction to the LLM, then validates candidates with WinGet search.	Safe
<code>Search WinGet</code>	Searches by name, task, or exact ID and writes a report.	Safe
<code>Run selected from last AI plan</code>	Opens the form for selected exact IDs from the last plan.	Dangerous for install/update/uninstall
<code>Add exact ID to list</code>	Adds a reviewed ID to a config list and GUI group.	Safe
<code>Install exact ID</code>	Installs one exact ID through the normal Audion path.	Dangerous
<code>Update exact ID</code>	Updates one exact ID through the normal Audion path.	Dangerous
<code>Uninstall exact ID</code>	Uninstalls one exact ID through the protected Audion path.	Dangerous

Dangerous actions use the standard Audion confirmation flow.

Building An AI Plan

When `Build plan` is clicked, the service:

1. Reads `AI task`.
2. Reads the prompt from `AI planner instruction` or the selected template.
3. Optionally scans installed WinGet IDs.
4. Optionally adds known Audion groups as context.
5. Calls the selected LLM provider.
6. Requires a JSON response following the package-plan schema.
7. Normalizes model suggestions.
8. Validates candidates with `winget search`.
9. Saves the last plan to cache.
10. Writes Markdown/JSON reports to `report\`.

The LLM returns a structured object with summary, packages, and notes.

Each package can include:

- display name;
- search query;
- proposed or exact WinGet ID;
- Audion group;
- intended action;
- reason;
- risk or review note.

WinGet ID Validation

The planner does not trust IDs only because the LLM suggested them.

Validation statuses:

Status	Meaning
<code>validated_exact</code>	Exact ID found through exact WinGet search.
<code>single_search_candidate</code>	No ID was provided, but search produced one strong candidate.
<code>search_candidates</code>	Candidates were found and need human review.

Status	Meaning
<code>id_not_found</code>	Suggested ID was not found.
<code>review_required</code>	Manual review is required.

Only usable exact IDs are exposed as checkboxes for running the AI plan.

Running Selected IDs From The AI Plan

The final flow opens from the top CTA `RUN SELECTED WITH AI` or the `Run selected from last AI plan` button in the action grid.

The form contains:

- `Action`;
- `Last AI plan exact IDs`;
- checkbox YAML import/export;
- final run button for the selected operation.

Available actions:

Action	Behavior
<code>Install missing</code>	Installs selected exact IDs through batch install.
<code>Update installed</code>	Updates selected exact IDs through batch update.
<code>Pin installed</code>	Adds IDs to pins and applies blocking pins.
<code>Check installed</code>	Checks selected IDs.
<code>Uninstall selected</code>	Uninstalls selected exact IDs through batch uninstall.

This path uses `winget_service._run_package_batch`, so it stays inside Audion's shared package-action logic.

Exact-ID Actions

The section also supports actions that do not require an LLM plan.

Search WinGet

`Search query` accepts:

- app name;
- vendor name;
- task;
- exact WinGet ID.

`Exact ID search` switches the search to exact mode.

Results are written to `report\<timestamp>_ai_search_winget\winget_search.md` and JSON.

Add exact ID to list

Adds a reviewed package ID to a selected list:

- `system`
- `dev`
- `ai`
- `pkms`
- `office`
- `media_images`
- `media_audio`
- `media_video`
- `network`
- `hardware`
- `custom`
- `pins`

For GUI-backed groups, the ID is added to `config\tool_manifest.yaml` so it appears as a normal checkbox.

Install exact ID

Runs the normal Audion install-by-ID path.

Update exact ID

Runs batch update for one ID.

Uninstall exact ID

Runs the normal protected uninstall-by-ID path.

Models

The `Models` tab manages provider and model selection. It does not install packages.

Fields:

Field	Purpose
<code>Model family</code>	Switches OpenAI/Gemini.
<code>OpenAI API key</code> / <code>Gemini API key</code>	Selects a key reference from config or cache.
<code>OpenAI model</code> / <code>Gemini model</code>	Selects a cached or API-discovered model.
<code>Model override</code>	Manual model ID if the model is valid but not in the list yet.
<code>OpenAI reasoning</code>	Reasoning effort for supported OpenAI models.
<code>LLM max output</code>	Response size limit.
<code>LLM retries</code>	Retry count after failures.
<code>LLM timeout</code>	Maximum wait for one request.

`Check model` sends a small request and stores the status in cache.

Caches And Local Files

File	Contents
<code>config\api_key_openai.txt</code>	Local OpenAI API key.
<code>config\api_key_gemini.txt</code>	Local Gemini API key.
<code>config\llm_settings.yaml</code>	Provider settings and defaults.
<code>config\gui_key_cache.json</code>	Favorite key references, no key material.
<code>config\gui_model_cache.json</code>	Model lists, favorites, and check statuses.
<code>config\gui_package_prompt_cache.json</code>	Saved prompt templates and pins.

File	Contents
config\gui_package_plan_cache.json	Last AI package plan.

API key material must not be written to GUI caches, logs, reports, or release archives.

Reports

AI Package Planner writes reports under `report\`:

- `ai_package_plan.md`
- `ai_package_plan.json`
- `winget_search.md`
- `winget_search.json`
- `action_summary.md`
- `action_summary.json`

Operation logs are written under `logs\`.

Safety Rules

Core rules:

- The LLM does not execute installation itself.
- The plan is shown to the user first.
- Only selected exact IDs are executed.
- Dangerous actions use Audion confirmation.
- WinGet IDs are validated through search before plan execution.
- Known Audion groups are not treated as installed state.
- API keys are not written to caches or reports.
- Protected uninstall remains inside the normal Audion protected flow.

Typical User Flow

1. Open `AI Package Planner`.
2. On `Planner`, fill `AI task`.
3. Keep or choose `Instruction template`.
4. Edit `AI planner instruction` only if needed.

5. Click **Build plan**.
6. Review the log and plan report.
7. Click **RUN SELECTED WITH AI**.
8. Choose an action and exact IDs from the last plan.
9. Confirm execution.

Typical DevOps Flow

1. Configure **config\api_key_openai.txt** or **config\api_key_gemini.txt**.
2. Check the model on **Models**.
3. Pin the working model and key.
4. Create and pin a prompt template for the workflow.
5. Build plans with **Build plan**.
6. Add reviewed IDs to project lists with **Add exact ID to list**.
7. Execute install/update only through Audion exact-ID operations.

Troubleshooting

Symptom	Check
Model does not answer	Key, selected model, timeout, max retries.
No models in list	API key, live model request, cache refresh.
Empty plan	AI task , prompt, LLM availability.
ID not found	Search WinGet , exact ID, source.
Package not offered for execution	Validation status, exact ID, last plan cache.
Prompt does not change	Selected Instruction template , cache refresh, clear button.

Developer Entry Points

File	Purpose
config\tool_manifest.yaml	Command, field, tooltip, and GUI group definitions.
system_core\ui_nicegui\app.py	Planner/Models tabs, action grid, top CTA, and pending forms.

File	Purpose
<code>system_core\services\winget_ai_service.py</code>	LLM calls, prompt/model/key cache, WinGet search validation, plan reports, exact-ID actions.
<code>system_core\providers\openai_provider.py</code>	OpenAI JSON/structured calls.
<code>system_core\providers\gemini_provider.py</code>	Gemini structured calls.
<code>system_core\services\winget_service.py</code>	Shared Audion install/update/pin/check/uninstall paths.

MCP/WinGet Boundary

The section is designed as an AI layer over WinGet: plan first, validate, let the user choose, then execute exact IDs.

Even as WinGet MCP support evolves, the safety model remains:

- plan first;
- validate exact IDs;
- user review;
- execute after approval;
- never bypass Audion package paths.

What WinGet itself ships today (checked against 1.29.280)

`winget mcp` prints the JSON snippet that wires `WindowsPackageManagerMCPServer.exe` into any MCP client. The server (`winget-mcp`) exposes exactly two tools:

- `find-winget-packages` — search and upgradeable listing (`query`, `upgradeable`), read-only;
- `install-winget-package` — install or upgrade (`identifier`, `source`, `upgradeOnly`), destructive.

Both are a subset of what Audion Get Tools already does directly: search for plan validation, and install/update by exact ID. Routing through MCP would drop the live ConPTY progress, the logs and reports, and the protected-ID check, so the planner keeps calling WinGet itself. `Health / Doctor` reports the MCP server path so it can be attached to an external MCP client without changing how the app runs packages.

Review Before Execution

For every selected package, verify exact ID, publisher, source, requested action, and whether the command affects the current user or the whole machine. Remove ambiguous candidates and packages already managed by another installer or corporate policy.

Use scan results to distinguish installed, available, upgradable, missing, and unknown IDs. Do not infer successful installation from a search result or planner selection.

After A Run

Review exit codes and the report for each ID. Some installers return success but require a reboot or interactive completion. Confirm the executable or registered package version when the item is important to an Audion runtime.

Keep failed IDs in a separate retry list with the original source and error. Rerun only after checking network, source availability, elevation, architecture, and installer conflicts.

GUI Manifest And Documentation

The GUI manifest defines the visible planners, package actions, fields, defaults, help text, and command bindings. This guide translates them into safe package-management decisions. When the manifest adds or renames an action, update the relevant workflow here and preserve exact-ID, source, elevation, review, and rollback cautions.

For a release check, compare the manifest labels with the GUI, command preview, generated package list, and final report. A friendly label must never conceal whether the backend searches, downloads, installs, upgrades, removes, or only stages a command.